

1. What does “salting” mean in the context of password hashing, and why is it used in Spring Security?

A: **Salting** means adding a unique, random string of data to a user's password *before* it is run through the hashing function. It is used to protect against **Rainbow Table attacks** and dictionary attacks. Because each user has a unique salt, even if two users choose the exact same password, their final hashed values will be different, making precomputed attacks ineffective.

2. What kind of information is typically stored inside a JWT (JSON Web Token)?

A: A **JWT** typically stores **claims**, which are pieces of information about the user or session. These include required claims like **iss** (issuer), **exp** (expiration time), and **sub** (subject, usually the user ID). It also often stores authorization claims, such as the user's **roles** or specific permissions. It is crucial never to store sensitive information like passwords in the payload because it is only signed, not encrypted.

3. What is SLF4J, and why is it commonly used for logging in Spring Boot applications?

A: **SLF4J (Simple Logging Facade for Java)** is not a logging framework itself; it is an **abstraction layer** or *façade*. It is commonly used because it decouples the application code from the logging implementation. This means you can write standard logging calls, and if you later decide to switch from the default **Logback** to a different system like Log4j2, you don't need to change your application's source code.

4. How does Spring Security protect an application from CSRF attacks, and in what situations is it acceptable or necessary to disable CSRF protection?

A: Spring Security protects against **CSRF (Cross-Site Request Forgery)** by using a unique, random value called a **CSRF token**. For any request that tries to change data (like POST or PUT), the token must be included in the request and match the token stored in the server's session. The attack fails because the malicious site cannot read or guess this secure token. It is acceptable to disable CSRF protection only for **stateless REST APIs** that use tokens (like JWT) for authentication, or for non-browser clients like mobile apps.

5. Your application has two roles: ADMIN and USER, and each role should access different REST endpoints. How would you configure Spring Security so that only specific roles can access certain URLs?

A: You configure this by defining security rules in your **SecurityFilterChain** bean. You use URL patterns combined with the **.hasRole()** method. For example, you would tell Spring: "Any

request to /admin/** must have the role ADMIN," and "Any request to /user/** must have the role USER." All other endpoints would require a user to simply be authenticated.

6. When a user tries to access a resource without sufficient permissions, you want to show a custom "Access Denied" page instead of the default Spring Security error response. How would you configure this behavior?

A: You configure this by providing a custom handler for access denied situations. You use the `.exceptionHandling()` configuration block and specify a custom page or URL using `.accessDeniedPage("/403-access-denied")`. Spring Security will automatically redirect the user to that URL whenever they receive a **403 Forbidden** error.

7. You need to secure a REST API built with Spring Boot. Describe, step by step, how you would secure it (authentication, authorization, stateless/session, CSRF, etc.).

A:

1. **Dependencies:** Add the Spring Security starter dependency.
2. **Stateless Config:** Disable CSRF and session management since REST APIs are usually stateless.
3. **Authentication:** Configure a custom filter to process the incoming **JWT token** to verify the user's identity.
4. **Authorization:** Define the security filter chain to allow or deny access to specific URLs based on the user's roles (`.hasRole('ADMIN')`).

8. What are the different options available in Spring Boot for securing REST APIs (e.g., basic auth, form login, JWT, OAuth2, API keys)?

A: The main options are **Basic Authentication** (sending username/password in headers), **Form Login** (session-based, common for traditional websites), **JWT** (stateless, token-based, best for APIs), and **OAuth2** (used for delegated authorization and external identity providers like Google or GitHub). **API Keys** can also be implemented using a custom filter.

9. How can you implement both authentication and authorization using Spring Security in a Spring Boot application?

A: **Authentication** is implemented by verifying user credentials (or a token) against a data source (like a database or in-memory list) to confirm their identity. **Authorization** is then implemented in the SecurityFilterChain by using methods like `.hasRole('ADMIN')` to allow or deny access to specific URLs based on the user's confirmed identity and roles.

10. If you must restrict certain REST endpoints based on roles (for example, ROLE_ADMIN vs ROLE_USER), which Spring Security configuration elements or annotations would you use?

A: I would primarily use URL matching in the security configuration file with the `.antMatchers()` (or `.requestMatchers()`) method, followed by `.hasRole('ADMIN')`. For checks inside the Java code, I would use the `@PreAuthorize` annotation directly on the service methods, such as `@PreAuthorize("hasRole('ADMIN')")`.

11. In a web application where users have different roles like ADMIN and USER, how would you implement role-based access control so that only admins can access management endpoints?

A: I would implement role-based control by using the `.authorizeHttpRequests()` method in the `SecurityFilterChain`. I would map the management endpoints (e.g., `/admin/**`) to require the ADMIN role using `.hasRole('ADMIN')`, while endpoints for regular users (e.g., `/user/**`) would require the USER role.

12. What is the difference between stateful session-based security and stateless token-based security in Spring, and when would you choose each for a REST API?

A: **Stateful security** (session-based) stores the user's authentication details on the server in a `HttpSession`. **Stateless security** (token-based, like JWT) stores all user data in a self-contained token on the client side. You choose **stateless** for REST APIs because it scales better, as the server doesn't need to track thousands of individual sessions.

13. How would you implement JWT-based authentication in a simple Spring Boot REST API (high level: login, token generation, filters, stateless config)?

A:

1. **Login:** A user logs in and the server generates a **JWT token**.
2. **Token Generation:** The token is signed with a secret key and sent back to the client.
3. **Stateless Config:** The security configuration is set to be stateless, disabling session creation.
4. **Filter:** A custom filter intercepts every request to read, validate, and process the token, setting the user's credentials in the security context.

14. What is a PasswordEncoder in Spring Security, and why is it mandatory to use one in modern applications?

A: A **PasswordEncoder** is an object used to securely hash (encrypt one-way) user passwords before they are stored in the database. It is mandatory because passwords must **never** be stored in plain text. Spring Security forces its use to protect applications from common attacks, like unauthorized access after a database breach.

JAVA Interview Buddy